

Integrating VLM for Autonomous Robotic Manipulation

Sauman Raaj
Boston University
sauman@bu.edu

Sreekar Praneeth Marri
Boston University
pramsml19@bu.edu

Anirudh S
Boston University
karthhik@bu.edu

Haroon Muhammed
Boston University
haroon@bu.edu

Abstract

This research explores the use of Vision-Language Models (VLMs) for robot manipulation using natural language. The project is divided into two phases, with Phase 1 evaluating basic motion control using a Large Language Model (LLM) alone, while Phase 2 integrated visual perception through a Visual Language Model (VLM) to help Sawyer with object manipulation tasks. The system demonstrated strong ability in allowing natural language to motion translation in simulation, paving the way for real-world deployment.

1. Introduction

Modern automation systems have generally relied on well-defined structured control protocols, by the use of state machines or pre-defined behaviors. While the techniques have specific purposes that work well, they limit the adaptability of robots within complex and dynamic environments. In contrast, natural language presents a simple process that allows people to interact without the need for robot programming or motion planning.

Recent advances in Large Language Models (LLMs) [9] and Vision-Language Models (VLMs) [7] have enabled the ability to map complex human commands to basic robot actions. For example, large language models as GPT-4 can understand and parse free-form text, while VLMs further enable grounding of language in visual contexts. In robotics, for example, these models allow users to command robots in the everyday language, specifying objects, spatial relations, and goals flexibly.

However, integrating LLMs and VLMs into real-world robotic control presents multiple challenges. These include the need for accurate perception, transformation of language-based goals into executable robotic motion instructions, also ensuring safety in dynamic environments. In this project, we explore these challenges by developing a modular framework wherein Sawyer robot, a 7-DOF

collaborative robotic arm [9], can interpret natural language instructions, perceive its environment visually, and plan safe trajectories to perform activities independently.

The major goal is to describe a complete pipeline that involves processing natural language input, through perception and reasoning, to motion planning and execution, setting the core phase for language-driven robotic manipulation in real-world applications.

The project was split into two separate phases:

1. **Phase 1:** Test text-to-motion capability using an LLM, interpreting commands like "move to (0.5, 0.2)."
2. **Phase 2:** Integrate vision by utilizing VLMs to locate objects and execute pick-and-place tasks in response to human instructions.

2. Literature Review

Recent advancements in vision-language models(VLM) have significantly enhanced the ability of robots to interpret natural language instructions in complex visual environments. However, most existing VLMs struggle with understanding spatial relationships such as "left", "behind", "closest to the blue object", which are critical for grounded robotic manipulation. To address this, SpatialVLM by Chen [2] enhances vision-language models with explicit spatial reasoning by training on a large-scale 3D spatial VQA dataset. It enables accurate interpretation of metric spatial relationships from 2D images, addressing a core limitation in conventional VLMs. This makes it a strong foundation for robotic systems that rely on spatially grounded language understanding for manipulation tasks.

Task-VLM [1] introduces a framework for task-oriented robotic manipulation by incorporating spatial relationships and object-level attributes like mass, fragility, and material. The model constructs spatial relation trees from visual and textual inputs, allowing structured reasoning over complex tasks. This enables more interpretable and physically informed manipulation decisions, complementing purely spa-

tial models like SpatialVLM .RoboPoint [14] further advances spatial grounding by predicting manipulation keypoints directly from language instructions. By learning to map commands to actionable 3D points, it eliminates reliance on symbolic reasoning and improves precision in real-world pick-and-place tasks. Physically Grounded VLMs[3] expand this reasoning by integrating physical object attributes such as weight, fragility, and material. This allows the model to infer safe and context-aware manipulation strategies, which are critical when interacting with objects that require delicate handling.

RoboFlamingo [6] builds on the OpenFlamingo architecture to adapt pre-trained VLMs for robotic manipulation through imitation learning. By fine-tuning on robot-specific data with minimal supervision it enables generalization to unseen manipulation tasks while maintaining efficiency. This approach demonstrates how lightweight adaptation of large VLMs can support real-world policy learning without end-to-end retraining. RobotGPT [4] introduces a framework in which a large language model, such as ChatGPT, is used to generate high-level task plans and code-based guidance. The generated output is transferred to a task specific network, which is trained to execute tasks reliably in real-time without relying on the original language model. This two-stage setup improves task consistency and reliability, particularly in long-horizon manipulation scenarios, while avoiding dependence on large-scale models during deployment. ChatGPT-Empowered Long-Step Robot Control [12] introduces a framework for leveraging large language models in multi-stage robotic planning through structured prompt templates. The method decomposes high-level language instructions into temporally ordered sub-tasks using few-shot in-context learning, enabling robust policy guidance across extended task sequences. By grounding prompts in visual observations and spatial context, the approach facilitates reasoning over long-horizon manipulation tasks without requiring task-specific fine-tuning. Vision-Language Model-Driven scene understanding and Robotics Object Manipulation [8] presents an integrated pipeline that combines visual perception, natural language understanding, and code generation for robotic manipulation in unstructured environments. The system employs a pre-trained VLM based on LLaMA2-chat to generate a semantic description of scenes, followed by zero-shot object grounding and pose estimation. These outputs are then passed to a code-generating language model that synthesizes high-level control commands, bridging the gap between perception, instruction, and execution in tabletop manipulation tasks.

CLIPort [10] introduces a two-stream architecture that combines CLIP-based semantic understanding with spatial precision from Transporter Networks. It performs zero-shot language-conditioned manipulation by aligning text prompts with dense pixel-level action maps, demonstrat-

ing strong performance in real-world pick-and-place tasks without requiring explicit object detection or symbolic planning. GPT-4V(ision) for Robotics [13] extends this direction by enabling robots to learn from one-shot human demonstrations using multimodal reasoning. The system combines visual analysis, symbolic task planning, and spatiotemporal grounding to extract affordances and generate executable manipulation strategies from demonstration videos, highlighting the potential of general-purpose VLMs in robot imitation learning.

3. Environment

3.1. Robot Operating System (ROS)

The Robot Operating System (ROS) is a flexible framework for developing robotic applications. It provides a plethora of tools, libraries, and conventions that aim to simplify the creation of complex and robust robot behaviors. ROS facilitates communication between different components of a robot system, such as sensors, actuators, controllers, and higher-level algorithms, through a distributed computing architecture. It offers various features, including message passing, service calls, parameter management, visualization tools, and simulation capabilities.

ROS is widely used in both research and industry across a diverse range of robotic applications, including but not limited to autonomous navigation, manipulation, perception, mapping, localization, and human-robot interaction. It has a large and active community that contributes to its development and maintenance, providing extensive documentation, tutorials, and software packages. ROS supports multiple programming languages, primarily C++, Python, and recently, Rust. Overall, ROS greatly accelerates the development and deployment of robotic systems by providing a standardized and modular framework that fosters collaboration and innovation in the robotics community. Its versatility and scalability make it an indispensable tool for researchers, developers, and engineers working on robotic projects of all scales and complexities.

3.2. Unified Robot Description Format(URDF)

URDF, or Unified Robot Description Format, is an XML-based file format used to describe robotic systems in a standardized and structured manner. It provides a way to represent the physical properties and geometry of robots, including links, joints, sensors, and visual and collision properties. URDF files are commonly used in conjunction with the Robot Operating System (ROS) to describe robots for simulation, visualization, and control. URDF files define the kinematic and dynamic properties of robots, allowing for accurate modeling and simulation of robot behavior. They enable visualization of robots in 3D environments and col-

lision checking for path planning and manipulation tasks. URDF files can also be used to specify robot configurations and launch files for ROS-based applications.

3.3. Gazebo Open Source Simulator

Gazebo serves as a comprehensive 3D simulator, capable of simulating both indoor and outdoor environments with multiple robots. It includes dynamic and kinematic physics simulations and supports various physics engines that can be easily plugged in. The integration of Gazebo with the Robot Operating System (ROS) is facilitated by a set of Gazebo plugins designed to support a wide range of existing robots and sensors. These plugins share a message interface consistent with the rest of the ROS ecosystem. This compatibility allows developers to write ROS nodes that work seamlessly with simulation, recorded data, and physical hardware. This means that you can create and test applications in the simulation environment and then deploy them to a real robot with minimal or no code changes.

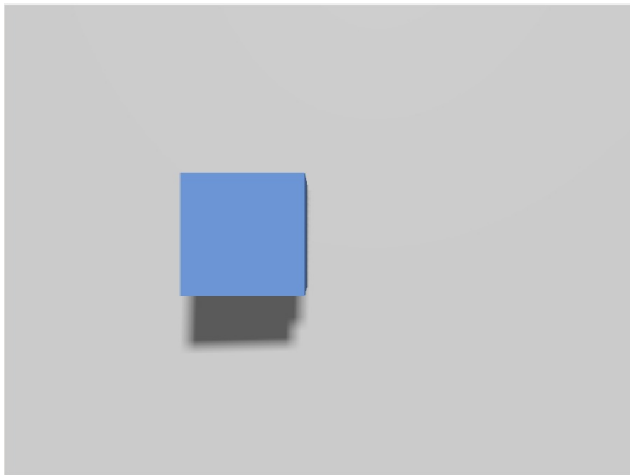


Figure 1. Image captured by Gazebo Simulator

4. Methodology

4.1. System Architecture

The architectural model in Figure 2 describes the complete process for implementing robotic applications using VLM for the Sawyer robotic arm. The process begins with a photo taken by the camera in the robot's head-mounted vision sensor. This image, along with a natural language instruction, is passed to the Vision-Language Model (VLM), which interprets the scene and the task. The VLM predicts a 2D pixel location (u, v) corresponding to the target object within the image. This 2D pixel is then combined with depth information to back-project the point into a 3D position in the camera frame. Subsequently, the 3D point is

converted into the robot's underlying coordinate system to provide a target location (x, y, z) in the real world. An inverse kinematics solver calculates a suitable joint configuration that would enable the end-effector to this target pose. MoveIt is used for trajectory planning to ensure collision-free and dynamically feasible motion. Finally, the generated motion plan is physically executed on the robot, producing the goal-directed manipulation task. This component-based architecture ensures that perception, reasoning, planning, and execution are cleanly decoupled while maintaining overall system robustness.

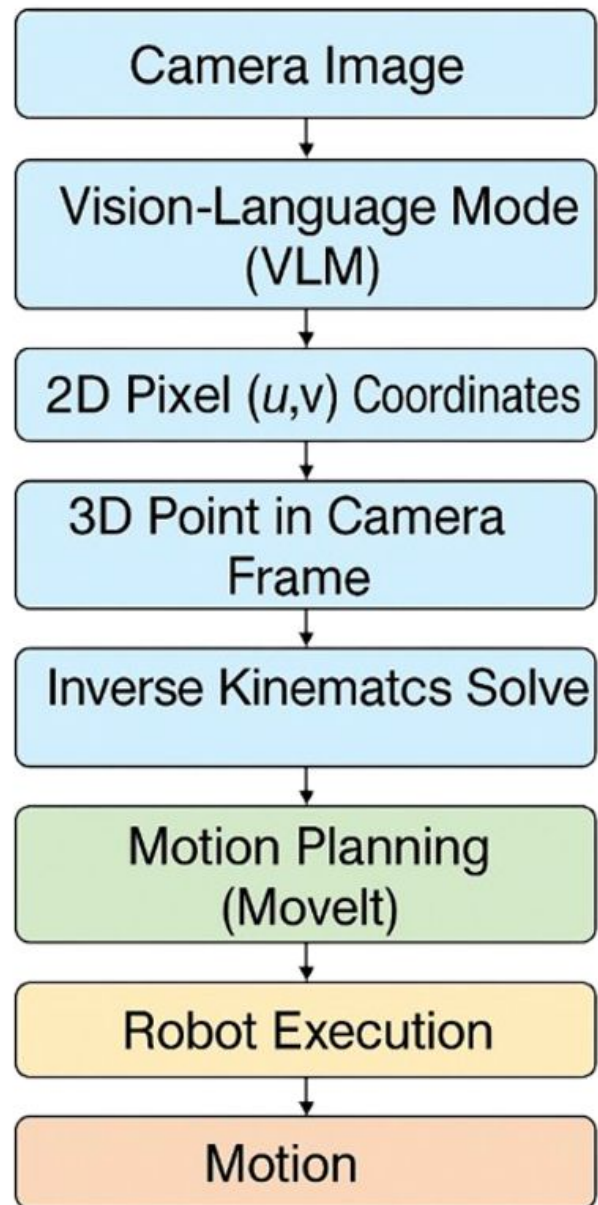


Figure 2. System architecture diagram for VLM-driven Sawyer control

4.2. LLM-Based Command Parsing

In the early stages of its development, a Large Language Model was utilized to parse natural language commands into structured robot instructions. When the LLM receives a user request, a preexisting JSON structure that includes the given actions, such as repositioning, closing, or engaging the gripping mechanism and lifting the object, are coupled actions linked to a specific API function call in the robotic control framework. The template for the prompt was carefully designed to ensure that the LLM consistently produces well-structured and executable outputs, even for varying natural language formulations. This method abstracts low-level control complexities from the user, enabling intuitive interaction and ensuring that the robot can execute the instructions without needing additional semantic parsing. Validation of command parsing through the use of LLMs was first performed, hence creating a strong foundation for extending the system to incorporate vision in the next phase.



Figure 3. Real Sawyer Robot Setup

4.3. Vision-Language Model Integration

In the next stage, the system was improved to incorporate visual perception by adding a Vision-Language framework. The Vision-Language Model (VLM) takes both a camera image along with a natural language directive as its input and predicts a 2D pixel location (u, v) of the target object in the image. Simultaneously, it outputs a structured system of activities necessary for the accomplish the objective, similar to the LLM-based model. This approach enables grounding of natural language commands

in visual observations, allowing the robot to independently detect objects and determine appropriate motion goals. A highly structured prompt instructs the VLM to produce consistent outputs, thus ensuring a uniform engagement with downstream processes coordinate transformation and action execution modules. The integration of visual input and linguistic processing enables the robot to interact more intelligently within unstructured spaces without being weighed down by prespecified object identifiers or manual target selection.

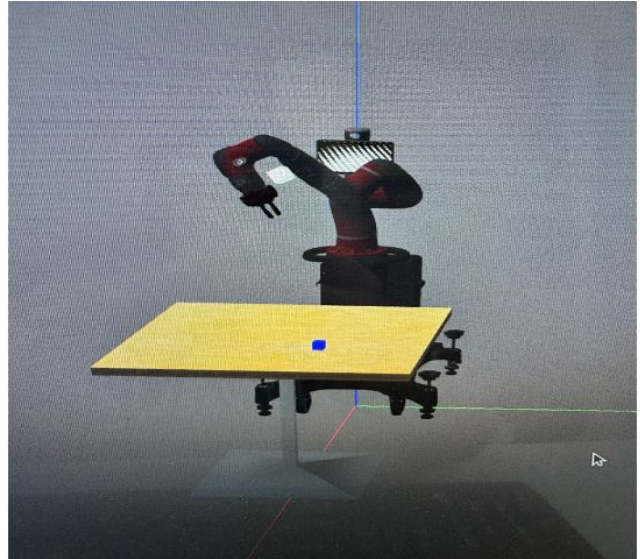


Figure 4. Sawyer Robot in Gazebo simulation environment

5. VLM Model Performance Comparison

Several Vision-Language Models (VLMs) were evaluated to determine their suitability for robotic system control. The tasks are rooted in two core measures: inference speed and localization accuracy. The models tested in The models released included GPT-4 Turbo [9], LLaVA 1.5 [7], Gemini 2.0 Flash, and Claude 3 Opus. Through the use of an image along with the corresponding verbal instructions, the object's expected coordinates were compared to the corresponding ground truth labels to evaluate the Euclidean localization error. In addition, average inference time per query was measured.

As indicated in Figure 6, the GPT-4 Turbo exhibited better localization because there was a lower Euclidean distance and has an error margin of 8.06 pixels along with a moderate inference latency of 3.22 seconds. LLaVA 1.5 provided faster reaction times (2.19 seconds), though reduced accuracy (27.73 pixels error). Gemini 2.0 Flash and The Claude 3 Opus showed a higher inference rate but

suffered from high localization inaccuracies making them unsuitable for precise manipulation tasks.

Since accurate localization of targets is crucial for successful robotic grasping, accuracy was given more priority over inference speed. GPT-4 Turbo was therefore selected as the base VLM architecture for this system, ensuring robust task execution even at the cost of slightly slower response times.

Model	Output	Ground Truth	Euclidean Error (px)	Inference Time (s)
GPT-4 Turbo	[449, 343]	[442, 347]	8.06	3.22
LLaVA 1.5	[417, 359]	[442, 347]	27.73	2.19
Gemini 2.0 Flash	[390, 420]	[442, 347]	89.63	1.03
Claude 3 Opus	[142, 142]	[442, 347]	363.35	1.60

Figure 5. Performance Comparison of Different VLM Models

5.1. Inverse Kinematics and Motion Execution

The system takes an RGB image from the robot’s head-mounted camera, which is then processed using a Vision-Language Model (VLM) to predict the coordinates of the center pixel (u, v), which is linked to target object. Simultaneously, a depth image from the camera provides depth information for each pixel, allowing the system to retrieve the depth d at the predicted (u, v) location. The 2D pixel location and the depth information is then incorporated using intrinsic camera calibration parameters to enable back-projection of the point into a 3D position in the camera coordinate frame.

To plan robot motion, the 3D point in the camera frame must be transformed into the robot’s base frame. This is achieved through TF2 pose transformations, thus ensuring all specified poses are properly expressed relative to the robot’s fixed coordinate system. The resulting 3D target pose (x, y, z) defines the desired location of the end-effector in the real -world environment.

An inverse kinematics solver, using MoveIt, selects a suitable joint configuration which achieves the target pose. In case a direct inverse kinematics solution is not possible, the system considers proximate points to set a realistic and achievable goal that builds resilience. Having determined an appropriate configuration of joints, MoveIt is further used to generate a complete collision-free trajectory, respecting joint limits, velocity and acceleration constraints, and obstacle avoidance.

At the higher level of task planning, the user instructions are converted by the VLM to structured form using a set

of designated application programming interface functions, such as `move to()`, `close gripper()`, and `open gripper()`, `lift()`. Those operations are interpreted and executed in real time dynamically, thus enabling the Sawyer robot to respond independently autonomously react to human language inputs.

```
def generate_prompt(self, instruction):
    return """You are a robot command generator.

    Given a user instruction, you must output a JSON object with:
    - "position": [x, y] image pixel coordinates (two floats)
    - "lift_height": how much to lift after picking (a float, in meters)
    - "actions": list of function names (with empty parentheses)

    Rules:
    - Always output valid JSON.
    - "position" must have exactly two floats.
    - "lift_height" must be a float.
    - "actions" must be a list of strings like "move_to()", "close_gripper()", "open_gripper()", "lift()".
    - Use only lowercase function names.
    - If no movement or position is needed, set position to [0.0, 0.0] and lift_height to 0.0.
    - Always complete all fields even if not used.

    ---

    Examples:

    Input: "Pick up the red cube"
    Output:
    {
      "position": [120, 240],
      "lift_height": 0.2,
      "actions": ["move_to()", "close_gripper()", "lift()"]
    }

    Input: "Grab the blue ball"
    Output:
    {
      "position": [180, 300],
      "lift_height": 0.25,
      "actions": ["move_to()", "close_gripper()", "lift()"]
    }
    """
```

Figure 6. Prompt template used for Vision-Language Model inference

6. Results

Both components of the project were successfully verified, i.e., LLM-based control and VLM-based control.

In Phase 1, the LLM-based system was deployed on the real Sawyer robot, where it reliably parsed natural language instructions like the one "move to (0.5, 0.2)" were converted to their corresponding actions using inverse kinematics and trajectory planning. The robot consistently reached the specified target poses without collisions. This well illustrates the viability of the language-to-motion model when applied to physical hardware.

In Phase 2, the VLM-based system was thoroughly tested in the Gazebo simulation environment. The system properly understood complex linguistic commands like "pick up the blue cube", through the implementation of the Vision-Language Model to detect objects and then design related motion trajectories. The robot showed the ability to independently identify target objects, define possible movement strategies, and execute pick-and-place actions, within the simulated environment, thus demonstrating the practicality of VLM-based robotics manipulation.

7. Conclusion and Future Work

This research demonstrated the feasibility of combining vision-language models with large language models to

achieve natural language-driven robotic control. Through a modular pipeline combining perception, reasoning, planning, and execution, the Sawyer robot was able to interpret high-level natural language instructions and autonomously perform motion planning towards designated destinations.

Future research will focus on implementing the system in its entirety on real-world hardware, along with improving the framework to deal with increasingly complex, multi-step manipulation tasks. Improvements in refining vision-language models for robotics-specific datasets [15], [11], [5] can notably enhance the reliability of perception. Furthermore, dynamic environment handling and real-time re-planning are important directions to expand the system's significance.

8. Contibution

The project was a collaborative effort where all four members participated, each offering unique inputs to different aspects of system development, testing, and documentation. The following are the details of the individual contribution:

1. **Sauman Raaj:** Worked on the integration of hardware elements with the Sawyer robot. This included implementing the LLM-based control system on the physical robot, establishing communication interfaces, and executing motion plans based on natural language commands. Also, led the setup of the real-world demonstration and actively contributed to the final presentation, focusing on showcasing empirical results from the hardware trials.
2. **Haroon Muhammed:** Lead the setup and configuration of the Gazebo simulation environment along with the MoveIt motion planning framework. Developed and validated the collision-free trajectory planning pipeline and inverse kinematics handling for the Sawyer robot in simulation. Generated visualizations and demo videos used in the report and presentation to illustrate system behavior.
3. **Anirudh S:** Developed and managed the ROS1-based control stack. Ensured seamless inter-module communication and maintained the integrity of the system codebase. Contributed significantly to documentation and presentation materials explaining the system pipeline.
4. **Sreekar Praneeth Marri:** Focused on the integration of Vision-Language Models (VLMs) into the robotic system. Implemented prompt engineering strategies, and handled the training and tuning of the LLM-based methods. Contributed extensively to the report and presentation by writing major sections and creating figures such as architectural diagrams and performance plots.

References

- [1] Nurhan Bulus Guran, Hanchi Ren, Jingjing Deng, and Xi-anhua Xie. Task-oriented robotic manipulation with vision language models. *arXiv e-prints*, pages arXiv–2410, 2024.
- [2] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14455–14465, 2024.
- [3] Jensen Gao, Bidipta Sarkar, Fei Xia, Ted Xiao, Jiajun Wu, Brian Ichter, Anirudha Majumdar, and Dorsa Sadigh. Physically grounded vision-language models for robotic manipulation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12462–12469. IEEE, 2024.
- [4] Yixiang Jin, Dingzhe Li, Jun Shi, Peng Hao, Fuchun Sun, Jianwei Zhang, Bin Fang, et al. Robotgpt: Robot manipulation learning from chatgpt. *IEEE Robotics and Automation Letters*, 9(3):2543–2550, 2024.
- [5] Goro Kobayashi, Tatsuki Kuribayashi, Sho Yokoi, and Kentaro Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- [6] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.
- [7] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [8] Sichao Liu, Jianjing Zhang, Robert X Gao, Xi Vincent Wang, and Lihui Wang. Vision-language model-driven scene understanding and robotic object manipulation. In *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, pages 21–26. IEEE, 2024.
- [9] J OpenAI Achiam, S Adler, S Agarwal, L Ahmad, I Akkaya, FL Aleman, D Almeida, J Altenschmidt, S Altman, S Anadkat, et al. Gpt-4 technical report. arxiv. *arXiv preprint arXiv:2303.08774*, 2023.
- [10] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pages 894–906. PMLR, 2022.
- [11] MB Vovchanskyi. A quick probability-oriented introduction to operator splitting methods. *arXiv preprint arXiv:2401.00123*, 2023.
- [12] Naoki Wake, Atsushi Kanehira, Kazuhiro Sasabuchi, Jun Takamatsu, and Katsushi Ikeuchi. Chatgpt empowered long-step robot control in various environments: A case application. *IEEE Access*, 11:95060–95078, 2023.
- [13] Naoki Wake, Atsushi Kanehira, Kazuhiro Sasabuchi, Jun Takamatsu, and Katsushi Ikeuchi. Gpt-4v (ision) for robotics: Multimodal task planning from human demonstration. *IEEE Robotics and Automation Letters*, 2024.
- [14] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024.
- [15] Hongbo Zhao, Andrej Košmrlj, and Sujit S Datta. Chemo-tactic motility-induced phase separation. *Physical review letters*, 131(11):118301, 2023.